

# Module 4: Computing (Scaling and Name Resolution)

**At the end of this module, you will be able to:**

- **Describe** Elastic Load Balancing features.
- **Differentiate** the types of ELB load balancers.
- **Describe** Amazon EC2 Auto Scaling and launch configurations.
- **Use** EC2 Auto Scaling in AWS.
- **Describe** Amazon Route 53 features and routing options.
- **Configure** failover routing.

## Topics:

- Elastic Load Balancing (ELB)
- Amazon EC2 Auto Scaling (including prediction challenge)
- Amazon Route 53

## Challenge:

- Leading the Automatic Scaling Prediction Challenge
- **Knowledge check**

## Recorded Demo:

- Amazon Route 53 routing options

## Lab:

- Lab 3: Configure Amazon EC2 Auto-Scaling

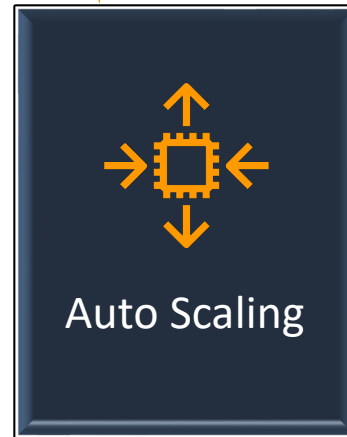
## Exercises / Project:

- Hands-on Activity 4: Configuring Failover Routing
- Project time: Troubleshooting knowledge base

# Troubleshooting knowledge base – Module 4 focus

- Have your copy of the troubleshooting knowledge base template open for editing
- **Create new entries** as you encounter troubleshooting situations
- **Categories of focus** for this module:
  - **Compute**
  - **Automation & Optimization**
  - **Networking**

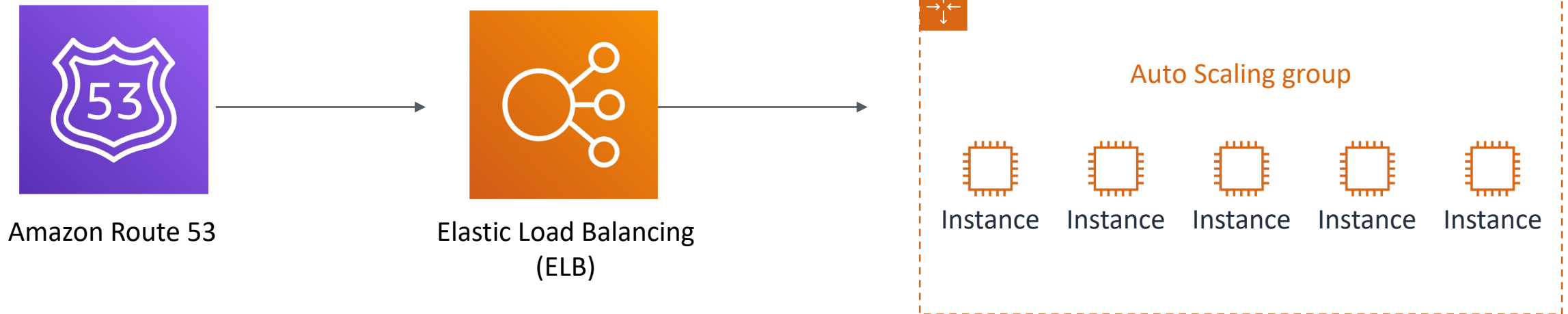
- Buying hardware to meet short-term demand creates waste
- Hardware is idle during off-peak periods and constrained during peak periods



## Scale on demand:

- Scale out during spikes
- Scale in during off-peak
- Replace unhealthy instances
- Pay only for what you use

# Components discussed



Module 4: Computing (Scaling and Name Resolution)

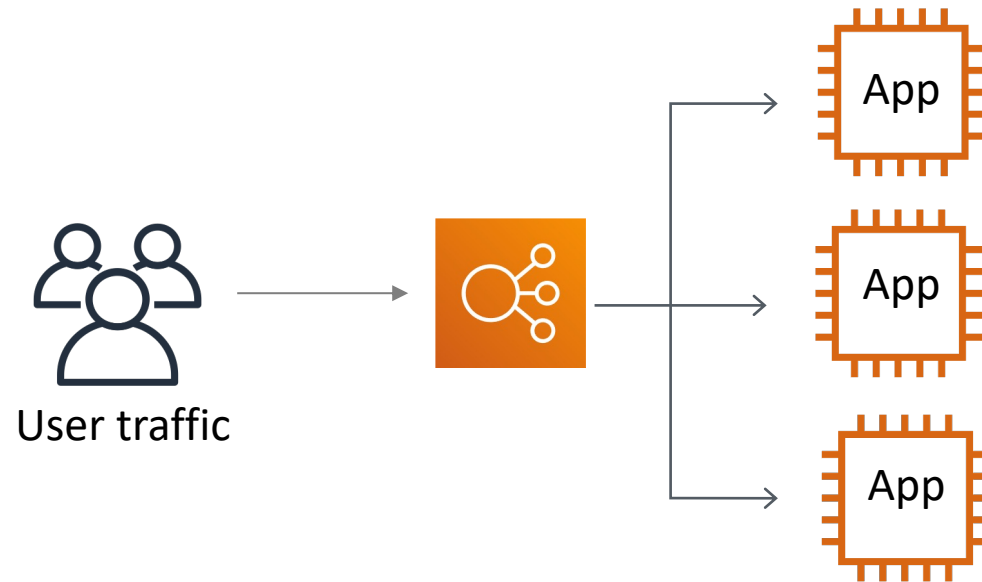
# Elastic Load Balancing (ELB)

# Elastic Load Balancing (ELB)



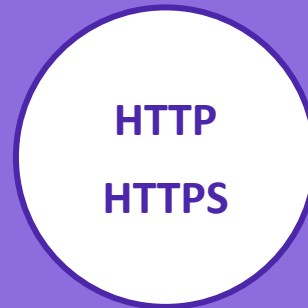
Elastic Load Balancing  
(ELB)

A **managed load balancing service** that distributes incoming application traffic across multiple Amazon EC2 instances, containers, and IP addresses.



# Type of load balancer: ALB

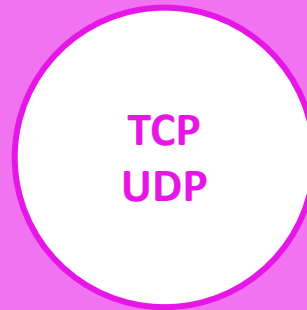
## Application Load Balancer (ALB)



- Flexible application management and supports static IP address
- Advanced load balancing of HTTP and HTTPS traffic
- Supports industry-standard protocols (WebSocket and HTTP/2)
- Operates at the application level (**Layer 7**)

# Type of load balancer: NLB

## Network Load Balancer (NLB)



- Extreme performance for your application
- Supports static IP address
- Load balancing of TCP or UDP traffic
- Operates at the connection level (**Layer 4**)

## Classic Load Balancer (CLB)

**PREVIOUS GENERATION**  
for HTTP, HTTPS, and TCP

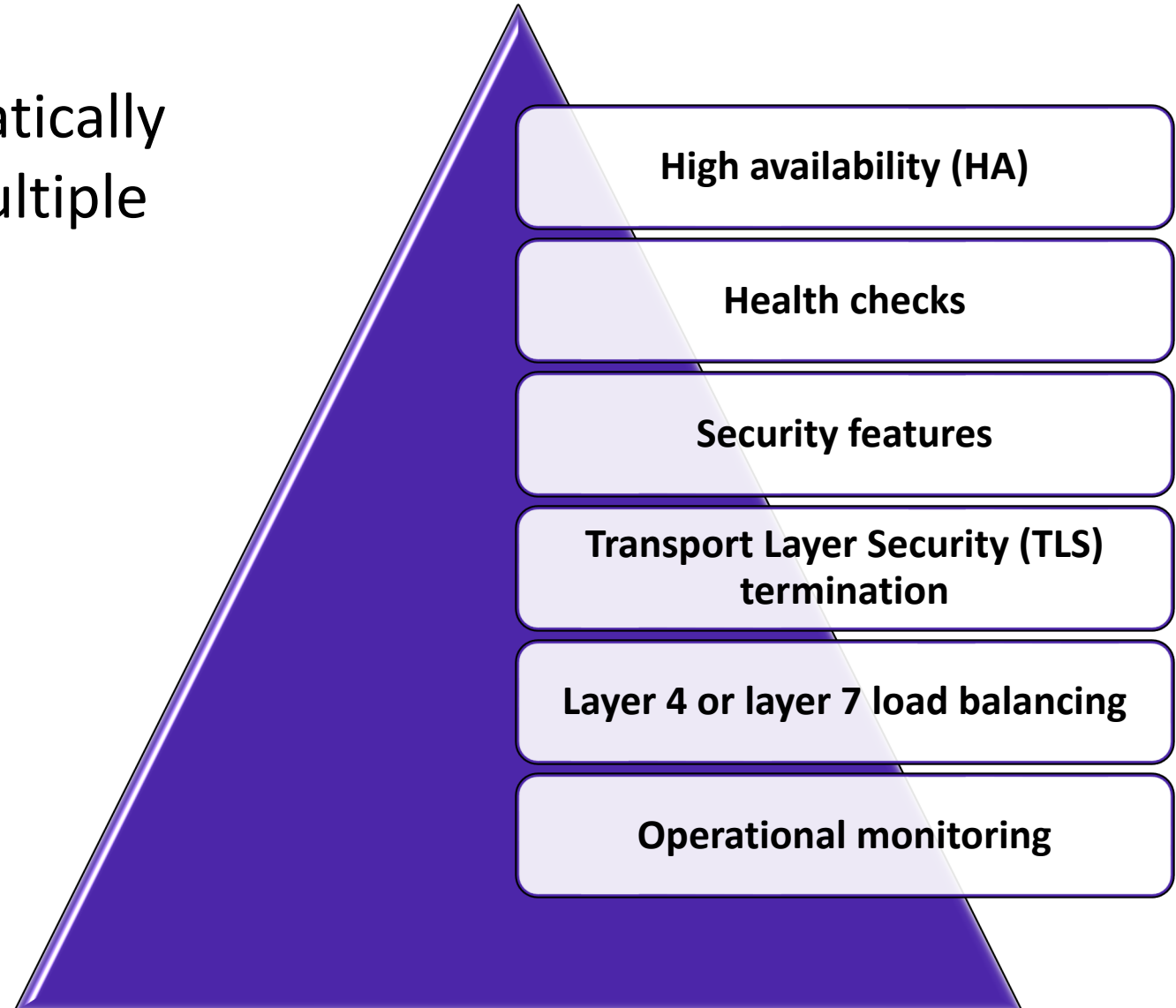
- For applications that use the EC2-Classic network
- Operates at both the request level and connection level

# ELB load balancer: Features

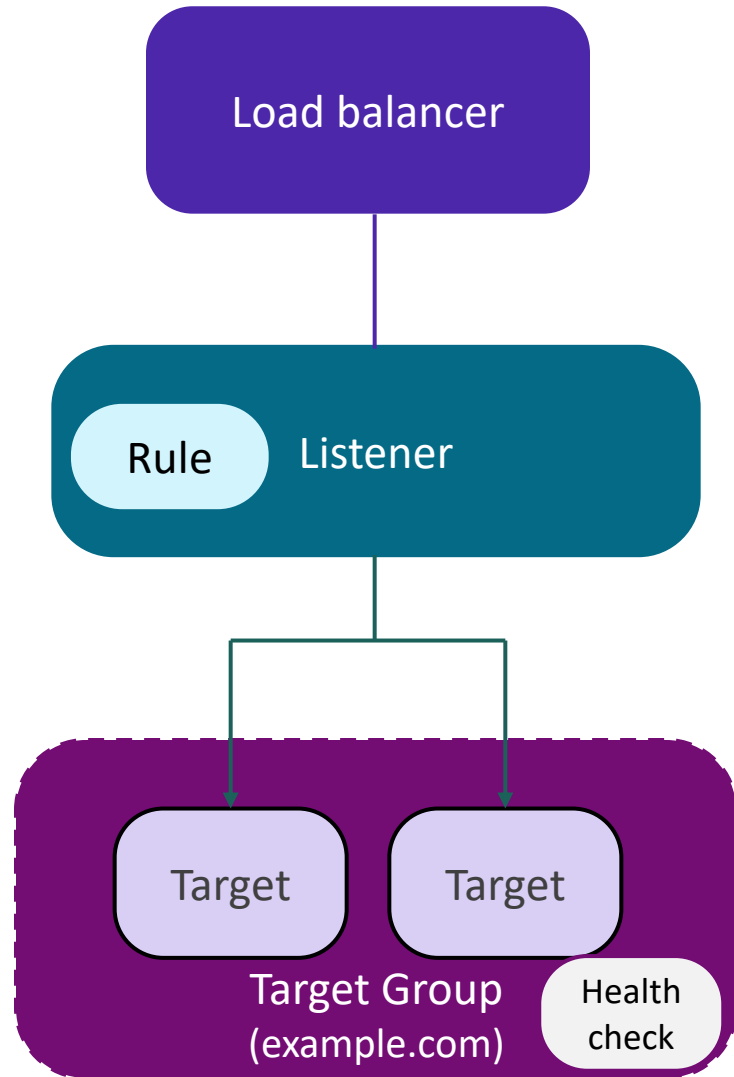
**ELB load balancers** automatically distribute traffic across multiple targets.



Elastic Load Balancing  
(ELB)



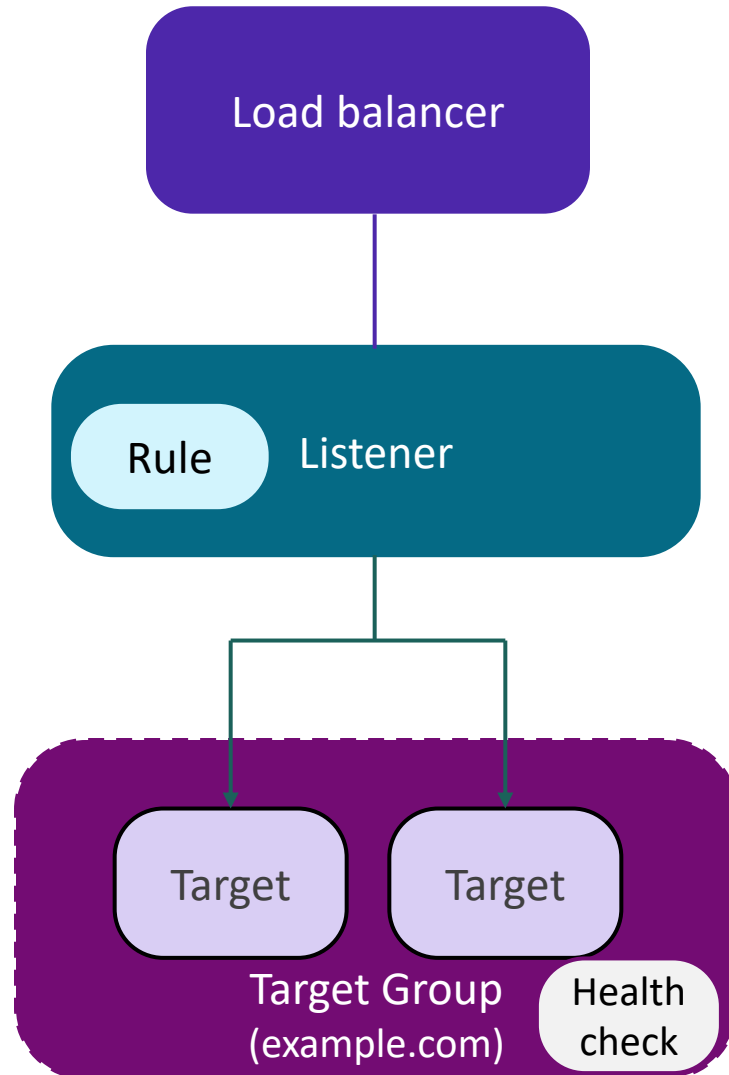
# Load balancer: Listeners



## Listeners:

- Process that defines the port and protocol that the load balancer listens on
- Each load balancer needs at least one listener to accept traffic
- Up to 50 listeners can be on a load balancer
- Routing rules are defined on listeners

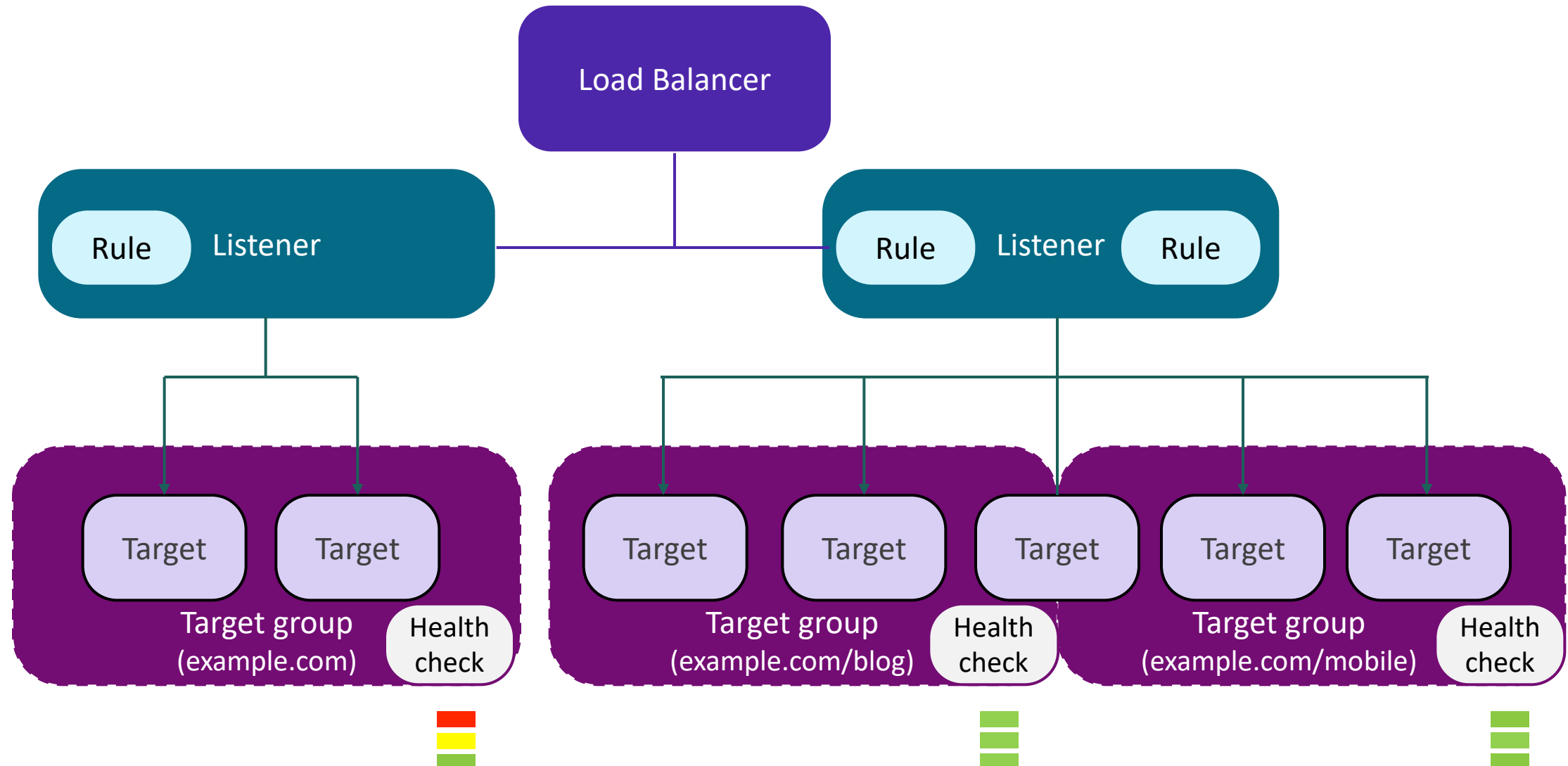
# Load balancer: Target groups



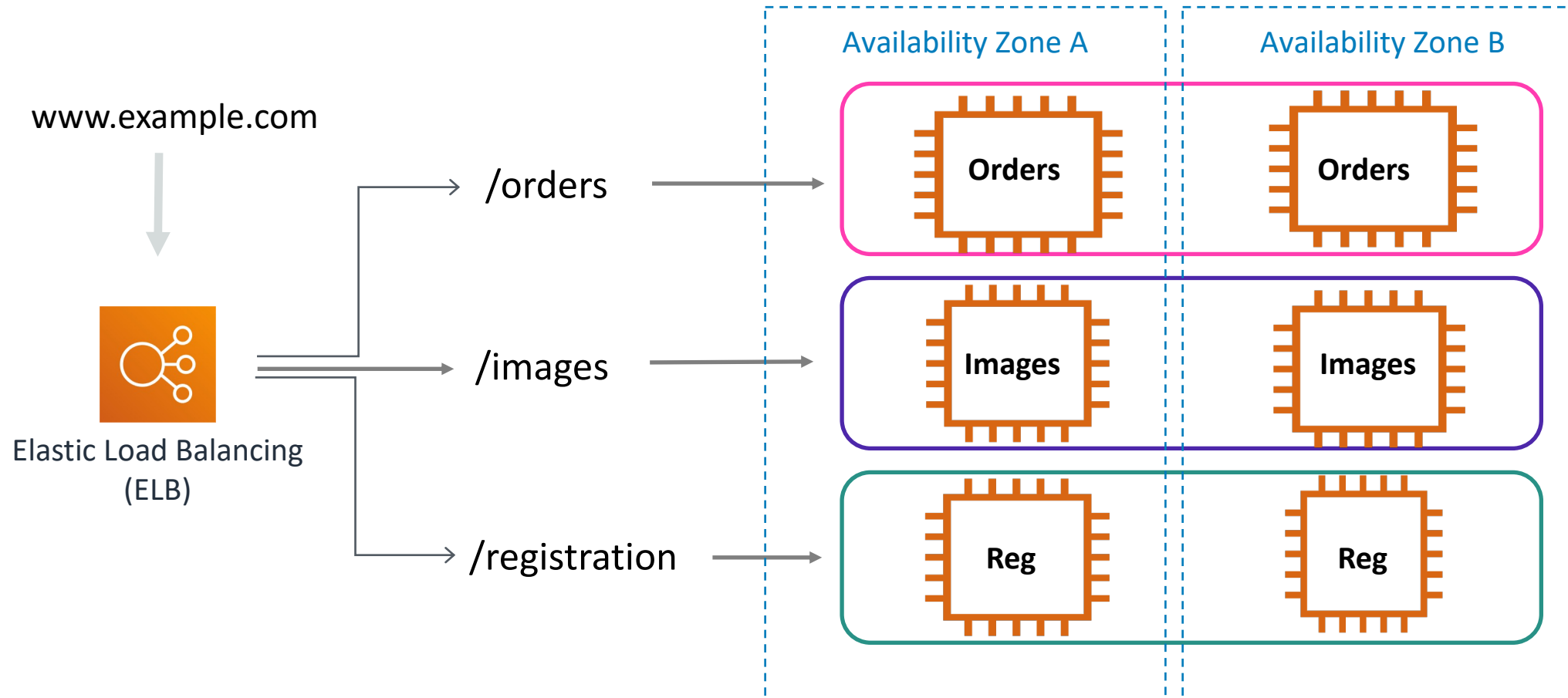
## Target groups:

- Group of registered targets that provide support to –
  - EC2 instances
  - ECS container instances
- A single target can have multiple target group registrations

# Application Load Balancer components



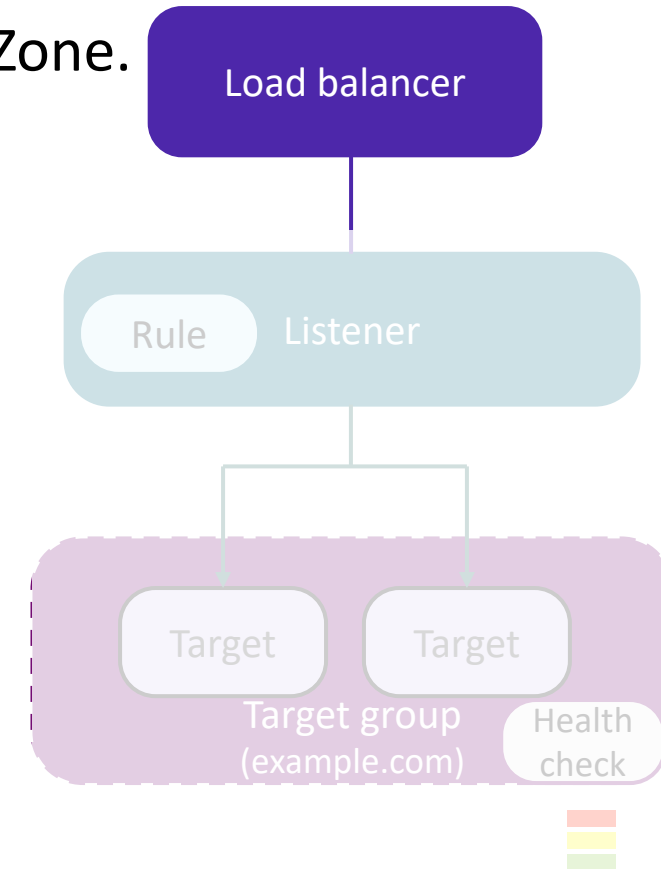
# Example Application Load Balancer



# Create an Application Load Balancer: AWS CLI

1. Use the **create-load-balancer command** to create a load balancer. You must specify two subnets that are not from the same Availability Zone.

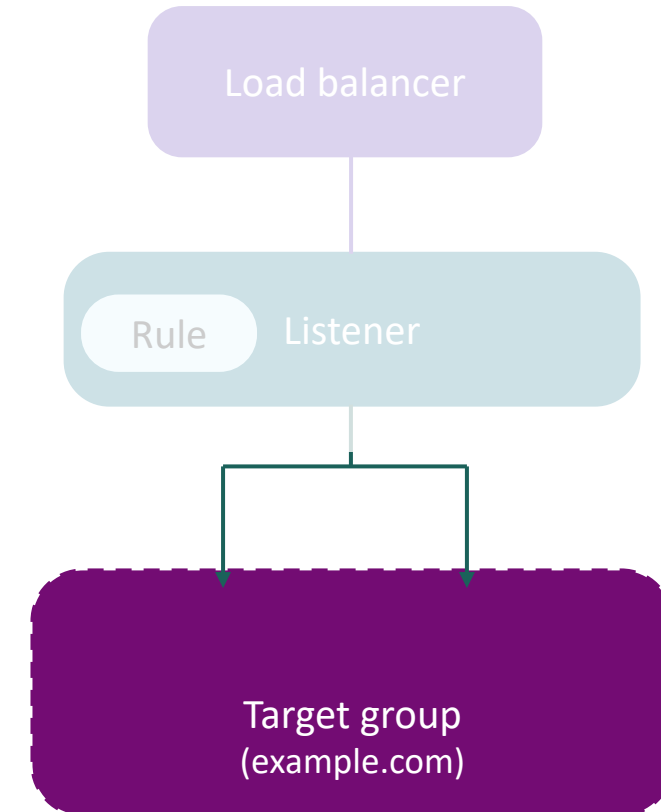
```
aws elbv2 create-load-balancer \  
--name my-load-balancer \  
--subnets subnet-12345678 subnet-23456789 \  
--security-groups sg-12345678
```



# Create a target group for the load balancer

2. Use the **create-target-group command** to create a target group. You must specify the Amazon VPC in which your EC2 instances are running.

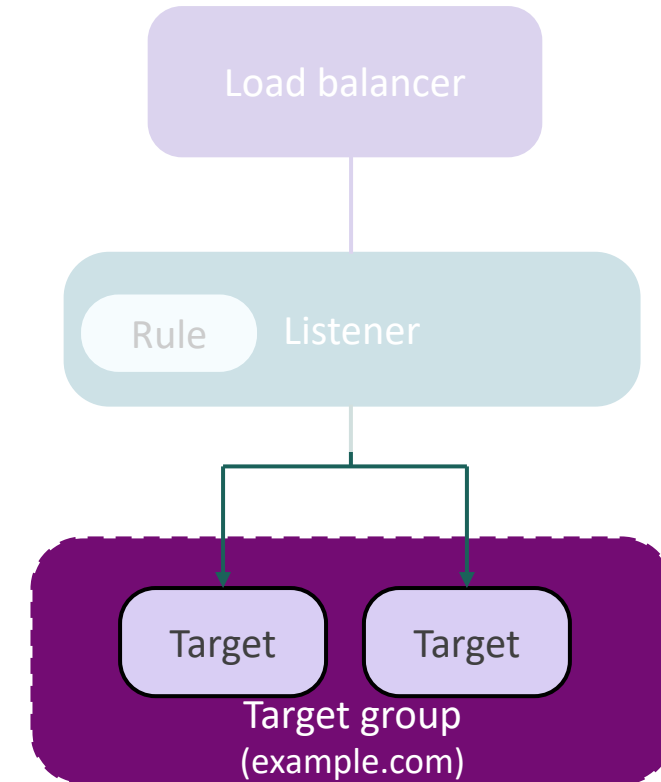
```
aws elbv2 create-target-group \  
--name my-targets \  
--protocol HTTP \  
--port 80 \  
--vpc-id vpc-12345678
```



# Register EC2 instances to the target group

3. Use the **register-targets command** to register your instances with your target group.

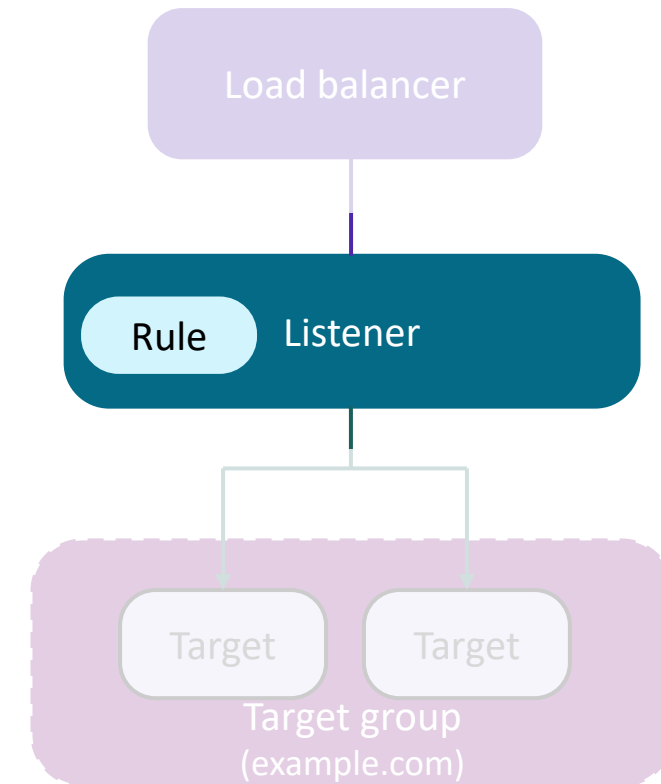
```
aws elbv2 register-targets \  
target-group-arn targetgroup-arn \  
--targets Id=i-12345678 Id=i-23456789
```



# Register EC2 instances to the target group

4. Use the **create-listener command** to create a listener for your load balancer. The listener has a default rule that forwards requests to your target group.

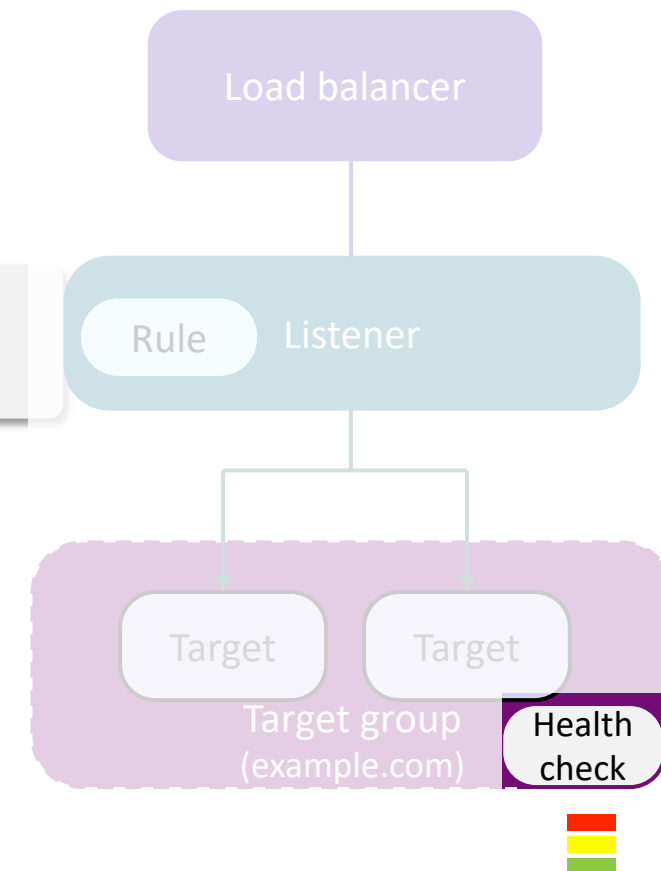
```
aws elbv2 create-listener \  
--load-balancer-arn loadbalancer-arn \  
--protocol HTTP \  
--port 80 \  
--default-actions Type=forward,TargetGroupArn=targetgroup-arn
```



# Verify the health of the registered targets

5. Optionally, verify the health of the registered targets for your target group using the **describe-target-health** command.

```
aws elbv2 describe-target-health --target-group-arn targetgroup-arn
```



## Key Takeaway



- There are three types of load balancers:
  - Application load balancers
  - Network load balancers
  - Classic load balancers
- Each listener checks for connection requests from clients and then forwards requests to one or more target groups, based on the defined rules.
- Each rule specifies a target group, condition, and priority.

Module 4: Computing (Scaling and Name Resolution)

# Amazon EC2 Auto Scaling

## Amazon EC2 Auto Scaling:



- Automatically launch or terminate Amazon EC2 instances based on:
  - Health status checks
  - User-defined policies that are driven by Amazon CloudWatch
  - Schedules
  - Other criteria (i.e., programmatically)
  - Manually using set-desired-capacity
- Scale out to meet demand, scale in to reduce costs

# EC2 Auto Scaling in action

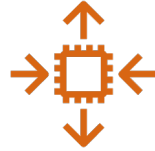
## Launch Configuration / Launch Template

Instance configuration to be launched.

Specify:

- AMI
- Instance type
- Security group
- Instance key pair
- Storage
- IAM roles
- User data

Only one active launch configuration at a time.



EC2 Auto Scaling

## EC2 Auto Scaling Group

Logical group of EC2 instances.

Automatically scale between:

- Minimum
- Desired (optional)
- Maximum

Integration with Elastic Load Balancing (optional).

Health checks to maintain group size.

Distribute and balance instances across Availability Zones.

## EC2 Auto Scaling Policy

Parameters for performing an Amazon EC2 Auto Scaling action.

How to trigger policies?

- Amazon CloudWatch
- Instance failure (health check)
- Scheduled
- Manually

Scale out or in, and by how much:

- ChangeInCapacity (+/-#)
- ExactCapacity(#)
- ChangeInPercent (+/-%)
- Cooldown period (simple scaling)
- Warmup period (step scaling)

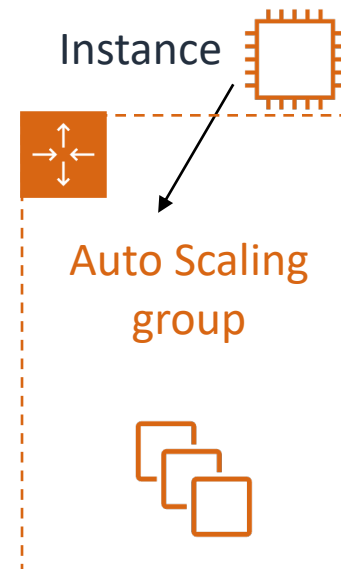
- Maintains health state for instances
- Terminates instances marked unhealthy
- Uses Amazon EC2 instance status checks by default
- Can configure these checks if instances are behind a load balancer:
  - Load balancer instance checks
  - Amazon EC2 instance checks
- Can trigger instance recycling by using external scripts:  
`aws autoscaling set-instance-health` command

# EC2 Auto Scaling termination policy

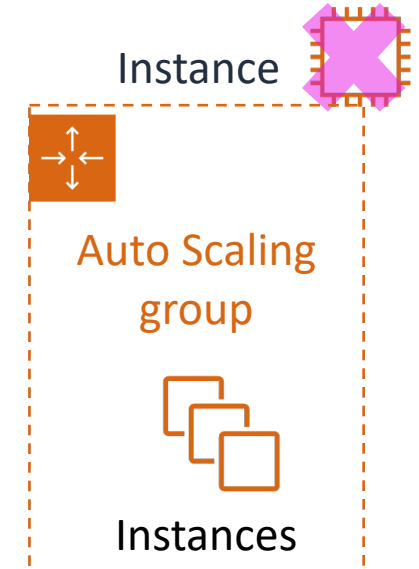
## Auto Scaling termination policy:

- Determines which instance is terminated when scaling in
- Determines termination order based on these factors:
  - Availability Zone with largest number of instances
  - Multiple policies (the policies will be run in the order they are listed)

### Scale Out New Instance Added



### Scale In Instance is Removed



# Termination policies

| Termination Policy                | Description                                                                |
|-----------------------------------|----------------------------------------------------------------------------|
| <b>OldestInstance</b>             | Selects longest-running instance                                           |
| <b>NewestInstance</b>             | Selects shortest-running instance                                          |
| <b>OldestLaunchConfiguration</b>  | Terminates instance with the oldest configuration <b>(default)</b>         |
| <b>ClosestToNextInstance Hour</b> | Terminates the instance closest to the next billable hour <b>(default)</b> |

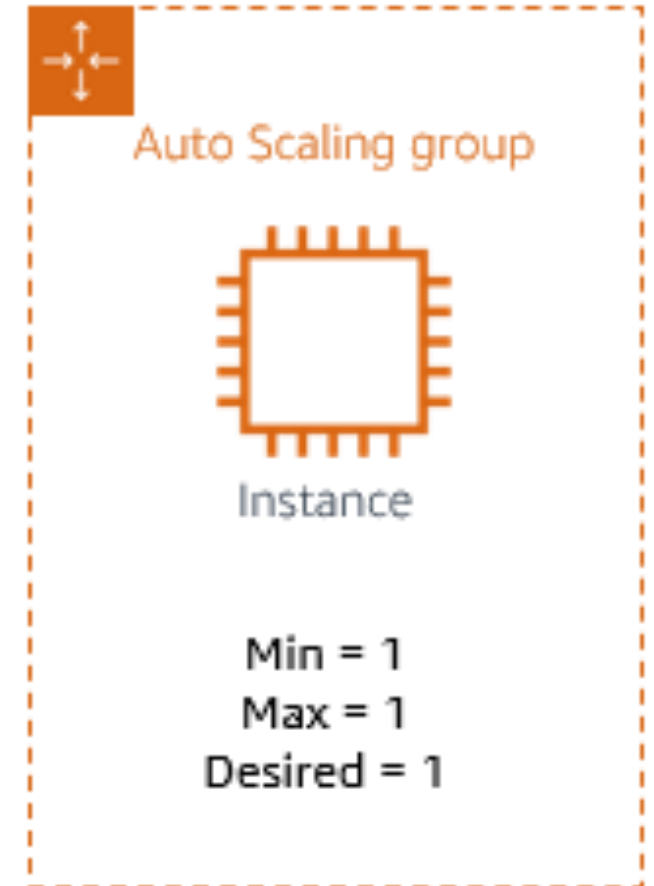
# Creating a steady-state group

## Steady-state group:

- You set an EC2 Auto Scaling group with the same min, max, and desired values
- Instance is recreated automatically if it becomes unhealthy or if Availability Zone fails
- Still potential downtime while instance recycles

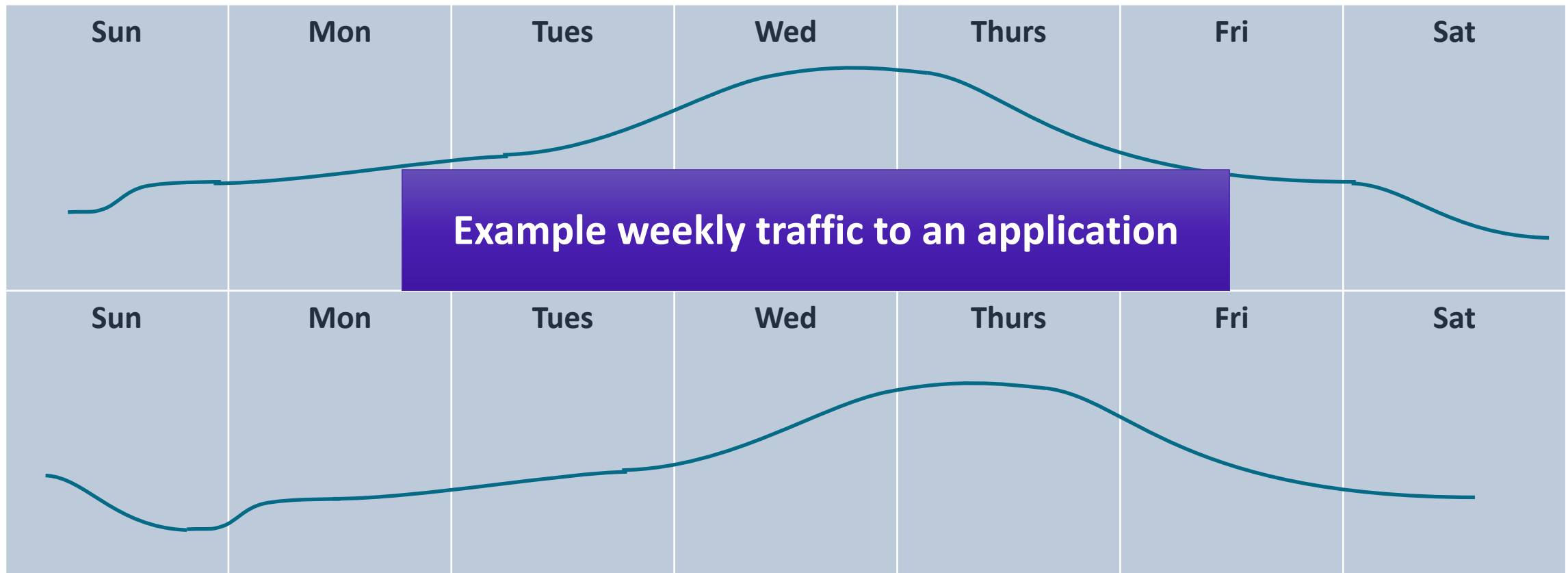
## Use case:

Maintain a steady-state NAT server in each Availability Zone.



# Scheduled scaling: Amazon EC2

Scaling that's based on a schedule allows you to scale your application in response to **predictable load** changes.



# Dynamic scaling: Amazon EC2

Increase or decrease the current capacity of the group based on a target value for a specific metric.

## Target Tracking Scaling

Increase or decrease the current capacity of the group based on a set of scaling adjustments, based on size of the alarm breach.

## Step Scaling

Increase or decrease the current capacity of the group based on a single scaling adjustment.

## Simple Scaling

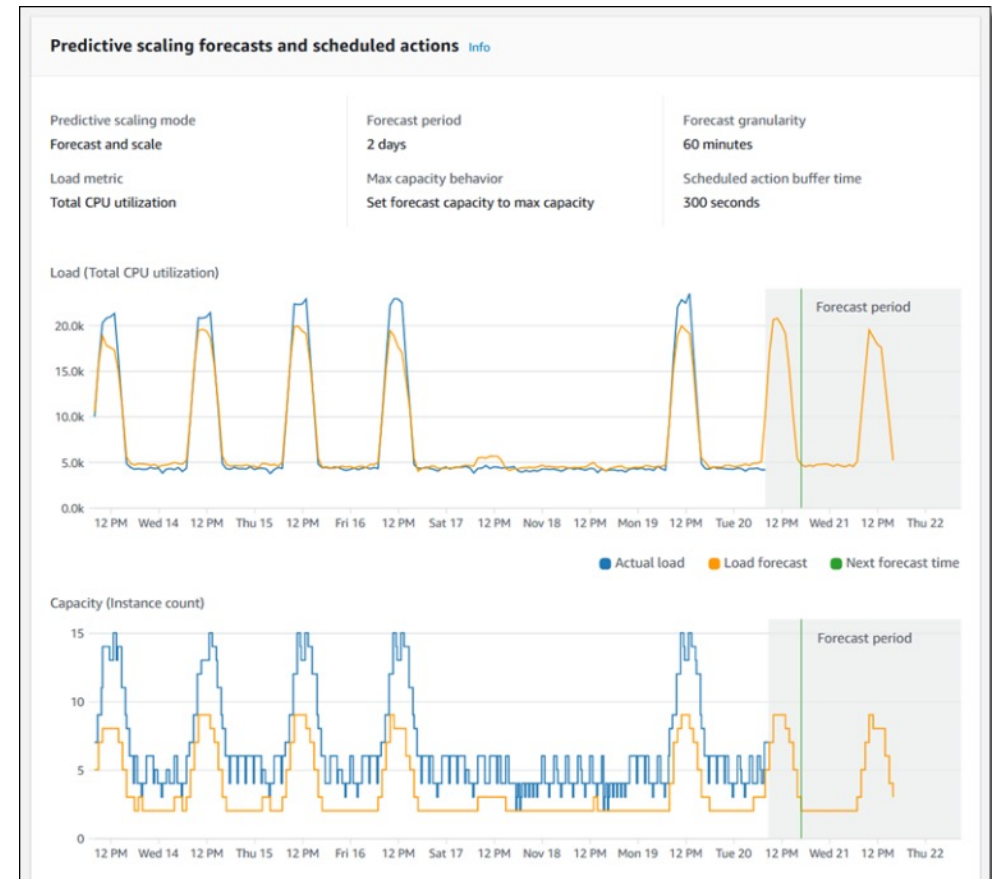
# Predictive scaling: Amazon EC2

## Predictive scaling:

- Forecast load
- Schedule minimum capacity

## Use case:

- Dynamic scaling target tracking
- Applications that have periodic spikes



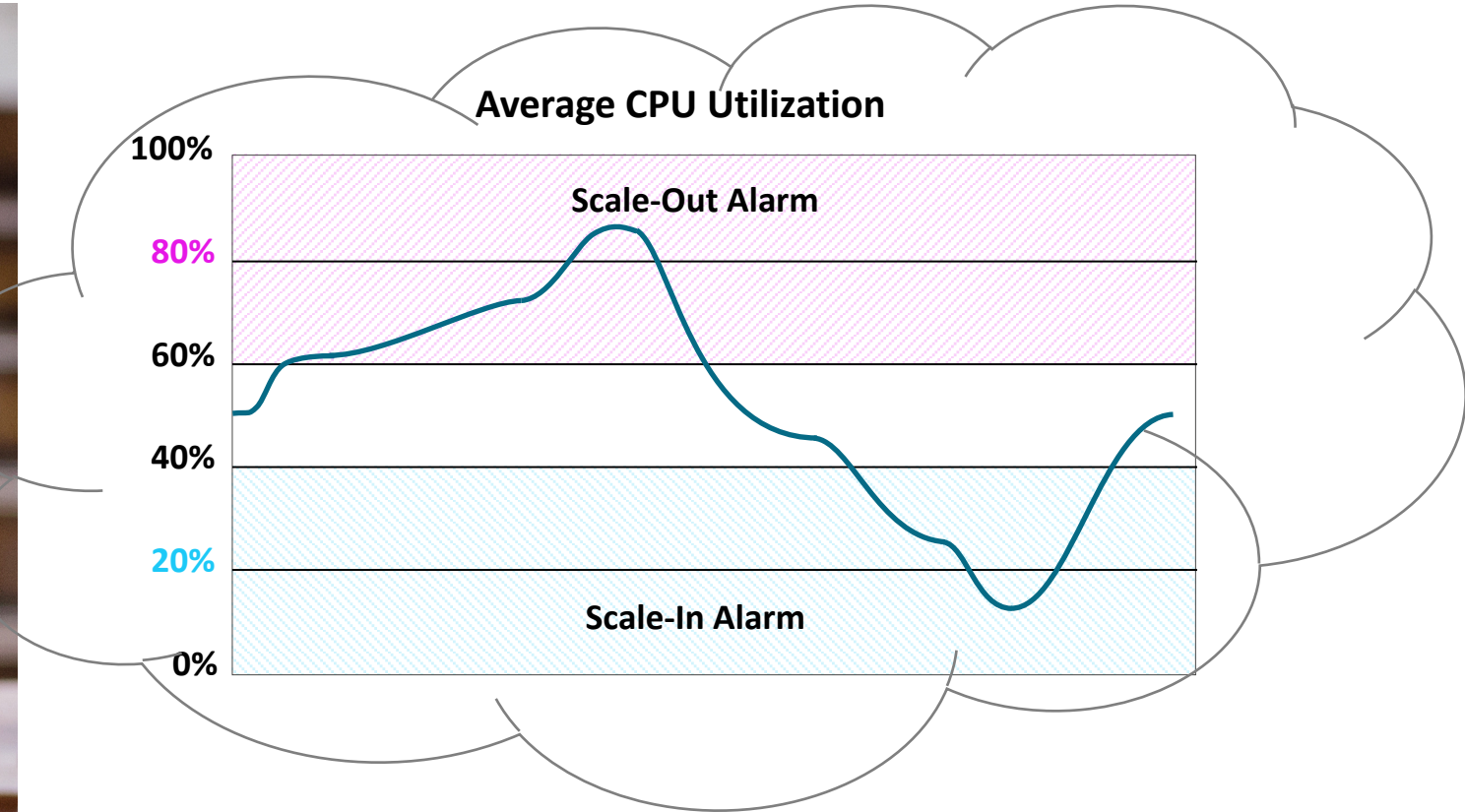
# Key Takeaway



- Amazon EC2 Auto Scaling helps maintain application availability by automatically adding or removing EC2 instances according to defined conditions.
- EC2 Auto Scaling consists of three parts:
  - Launch configuration/launch template
  - Auto Scaling group
  - Scaling policies
- Scaling can be based on:
  - Instance health
  - Amazon CloudWatch alarms
  - Time schedule or past usage (prediction)
- To create a launch configuration, several details must be specified.

# Challenge: Leading the Automatic Scaling prediction challenge

# Sophie needs the ability to predict



# Challenge: Condition #1

## Step scaling policy alarms:

- **Scale-in:** Remove one instance when  $40\% > \text{average CPU} > 20\%$  for more than 2 minutes
- **Scale-in:** Remove two instances when  $20\% > \text{average CPU} > 0\%$  for more than 2 minutes
- **Scale-out:** Add one instance when  $60\% < \text{average CPU} < 80\%$  for more than 2 minutes
- **Scale-out:** Add two instances when  $80\% < \text{average CPU} < 100\%$  for more than 2 minutes

For each condition, predict the number of instances that are added or removed.

**Condition #1:** Average CPU use has been 63–70% for more than 2 minutes.

***Note:** Assume 10 instances to start and the instance warmup period is set to 5 minutes.*

# Challenge: Condition #2

## Step scaling policy alarms:

- **Scale-in:** Remove one instance when  $40\% > \text{average CPU} > 20\%$  for more than 2 minutes
- **Scale-in:** Remove two instances when  $20\% > \text{average CPU} > 0\%$  for more than 2 minutes
- **Scale-out:** Add one instance when  $60\% < \text{average CPU} < 80\%$  for more than 2 minutes
- **Scale-out:** Add two instances when  $80\% < \text{average CPU} < 100\%$  for more than 2 minutes

For each condition, predict the number of instances that are added or removed.

**Condition #2:** The CPU pressure continues to build and is holding at 60–80%. It has been 2 minutes since Condition #1 occurred.

**Note:** Assume 10 instances to start and the instance warmup period is set to 5 minutes.

# Challenge: Condition #3

## Step scaling policy alarms:

- **Scale-in:** Remove one instance when  $40\% > \text{average CPU} > 20\%$  for more than 2 minutes
- **Scale-in:** Remove two instances when  $20\% > \text{average CPU} > 0\%$  for more than 2 minutes
- **Scale-out:** Add one instance when  $60\% < \text{average CPU} < 80\%$  for more than 2 minutes
- **Scale-out:** Add two instances when  $80\% < \text{average CPU} < 100\%$  for more than 2 minutes

For each condition, predict the number of instances that are added or removed.

**Condition #3:** While less than 5 minutes have passed since Condition #1 occurred, CPU use is now at 85%.

***Note:** Assume 10 instances to start and the instance warmup period is set to 5 minutes.*

# Challenge: Condition #4

## Step scaling policy alarms:

- **Scale-in:** Remove one instance when 40% > average CPU > 20% for more than 2 minutes
- **Scale-in:** Remove two instances when 20% > average CPU > 0% for more than 2 minutes
- **Scale-out:** Add one instance when 60% < average CPU < 80% for more than 2 minutes
- **Scale-out:** Add two instances when 80% < average CPU < 100% for more than 2 minutes

Predict the number of instances that are added or removed.

**Condition #4:** More time has passed. The first of the two added instances is warmed up and handling part of the load. The second instance is still in a warmup state. But CPU use has dropped to about 53% for the last 3 minutes.

# Challenge: Condition #5

## Step scaling policy alarms:

- **Scale-in:** Remove one instance when 40% > average CPU > 20% for more than 2 minutes
- **Scale-in:** Remove two instances when 20% > average CPU > 0% for more than 2 minutes
- **Scale-out:** Add one instance when 60% < average CPU < 80% for more than 2 minutes
- **Scale-out:** Add two instances when 80% < average CPU < 100% for more than 2 minutes

Predict the number of instances that are added or removed.

**Condition #5:** Now with reduced demand on the application deployment, the CPU use has dropped to a steady 32%.

# Challenge: Condition #6

## Step scaling policy alarms:

- **Scale-in:** Remove one instance when 40% > average CPU > 20% for more than 2 minutes
- **Scale-in:** Remove two instances when 20% > average CPU > 0% for more than 2 minutes
- **Scale-out:** Add one instance when 60% < average CPU < 80% for more than 2 minutes
- **Scale-out:** Add two instances when 80% < average CPU < 100% for more than 2 minutes

Predict the number of instances that are added or removed.

**Condition #6:** CPU use has gone down even more and is now at about 17%.

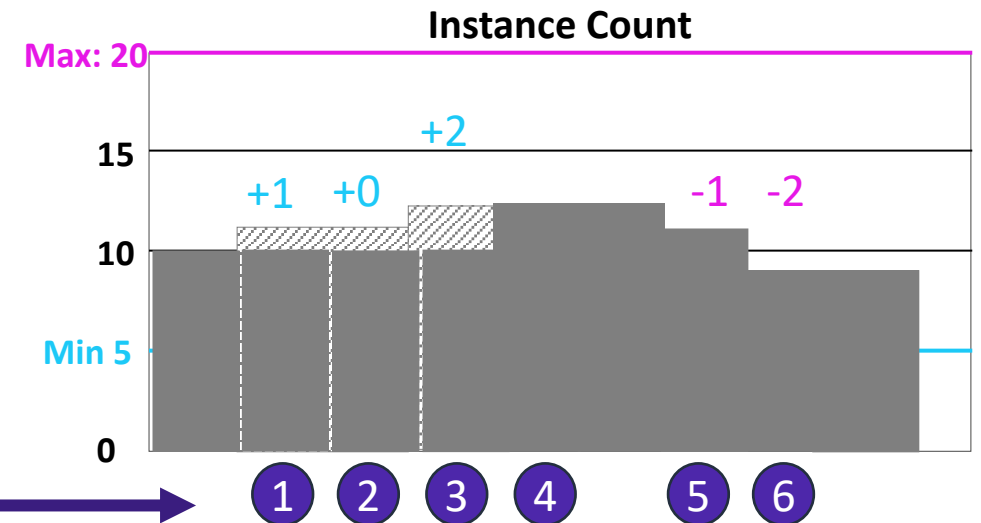
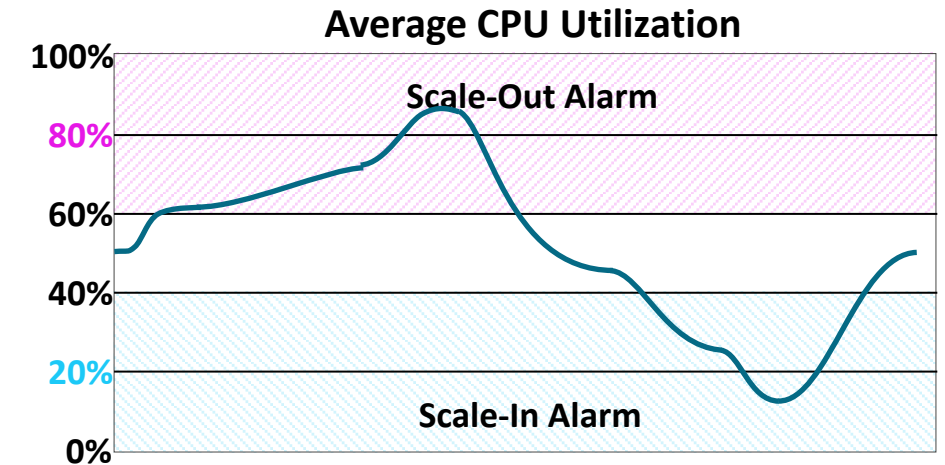
# Step scaling policy challenge: Answer key

*Did your predictions match the results shown in these two graphs?*

## Step scaling policy alarms:

- **Scale-in:** Remove 1 instance when  $40\% > \text{average CPU} > 20\%$  for more than 2 minutes
- **Scale-in:** Remove 2 instances when  $20\% > \text{average CPU} > 0\%$  for more than 2 minutes
- **Scale-out:** Add 1 instance when  $60\% < \text{average CPU} < 80\%$  for more than 2 minutes
- **Scale-out:** Add 2 instances when  $80\% < \text{average CPU} < 100\%$  for more than 2 minutes

*When the condition numbers in the challenge occurred*





## Alarm sustain period:

**Example:** CPU utilization > 90% for **10 minutes**



## Cooldown period:

- When running a scale in or out, suspend further scaling activities for a cooldown period of time
- Used for simple scaling policies

**Example:** Suspend scaling for **5 minutes**

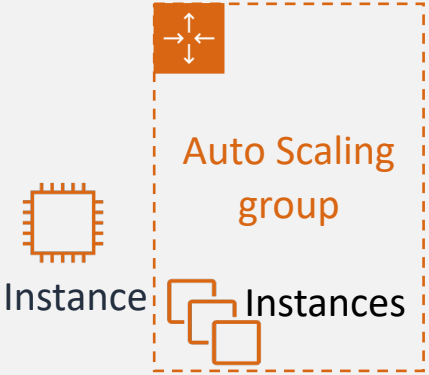
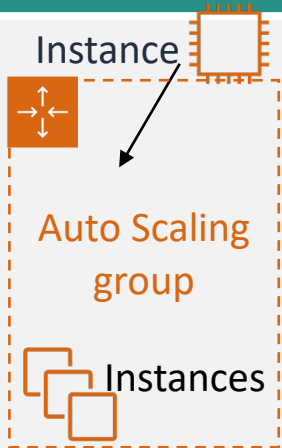
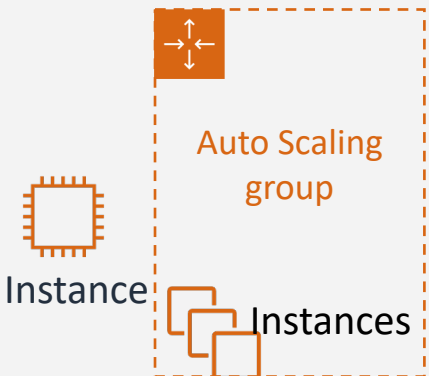


## Instance warmup period:

- The number of seconds that it takes for a newly launched instance to warm up
- Used for step scaling policies

**Example:** Warmup takes **5 minutes**

# EC2 Auto Scaling group lifecycle hooks

| Event     | Before                                                                             | Auto Scaling Action                                                                                                                  | User/Programmed Action                                                               |                                                                                                                                         | After                                                                                |
|-----------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Scale Out |   | <ol style="list-style-type: none"><li>1. <b>Launch</b> the instance.</li><li>2. Send notification.</li></ol>                         |   | <ol style="list-style-type: none"><li>3. Act on the instance (e.g., install software).</li><li>4. <b>Add</b> it to the group.</li></ol> |   |
| Scale In  |  | <ol style="list-style-type: none"><li>1. <b>Remove</b> instance from the Auto Scaling group.</li><li>2. Send notification.</li></ol> |  | <ol style="list-style-type: none"><li>3. Act on the instance (e.g., retrieve logs)</li><li>4. <b>Terminate</b> it.</li></ol>            |  |

Module 4: Computing (Scaling and Name Resolution)

# Amazon Route 53

## Amazon Route 53:



Amazon Route 53

- Is a scalable Domain Name System (DNS) web service
  - Distributes traffic across Regions
    - Can support high availability and lower latency
  - Register or transfer a domain name
  - Resolve domain names to IP addresses
  - Route requests based on latency, health checks, weights, etc.

# Associating DNS names with ELB

- Default DNS name from ELB:
  - AWS assigns a hostname to your load balancer that resolves to a set of IP addresses

## **Example:**

web-app.us-west-2.elb.amazonaws.com

- Associate your own DNS name:
  - Assign your own hostname via an alias resource record set
  - Create a CNAME record that points to your load balancer

## **Example:**

example.com → ALIAS → web-app.us-west-2.elb.amazonaws.com

# Amazon Route 53 routing policies

Amazon Route 53 supports these seven routing policies:

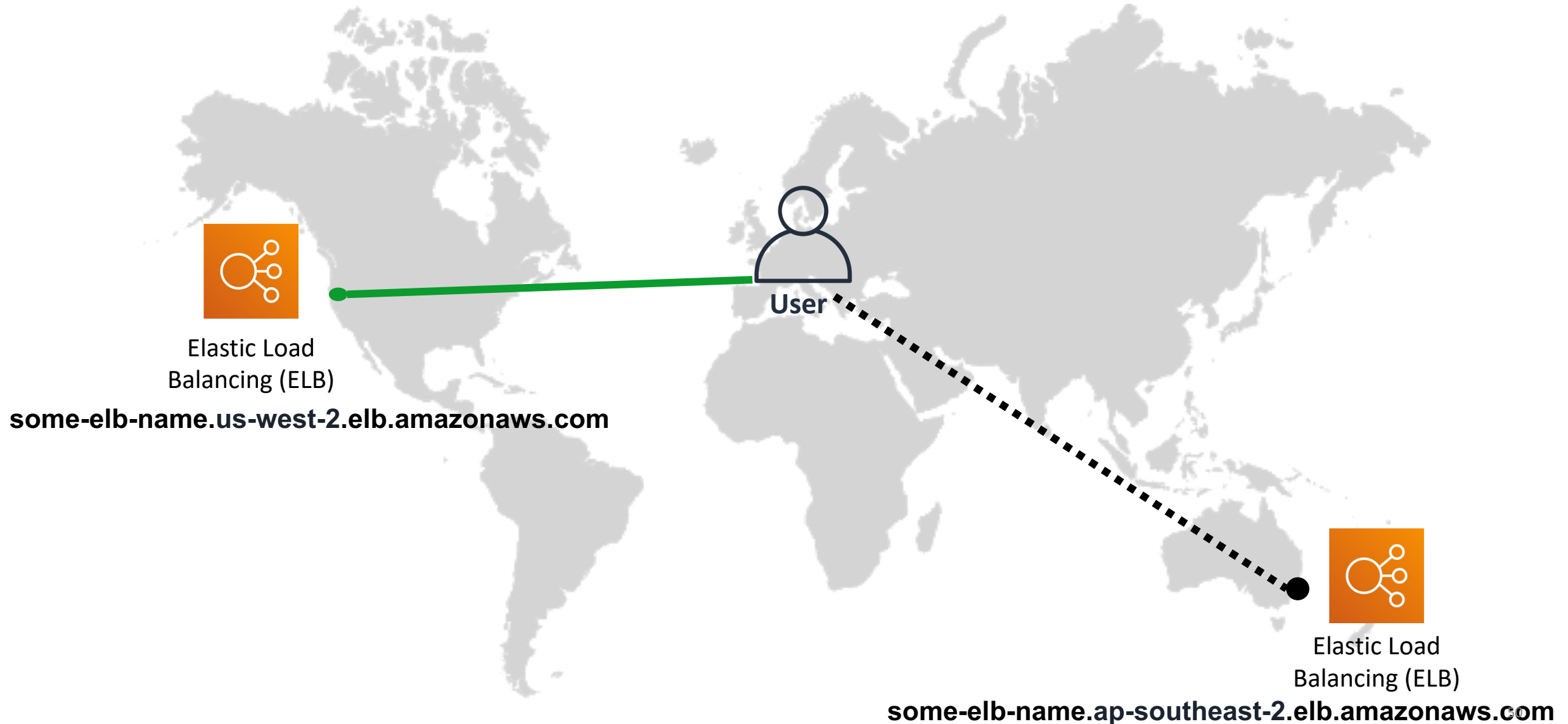


1. Simple routing policy
2. Weighted routing policy
3. Latency routing policy
4. Failover routing policy

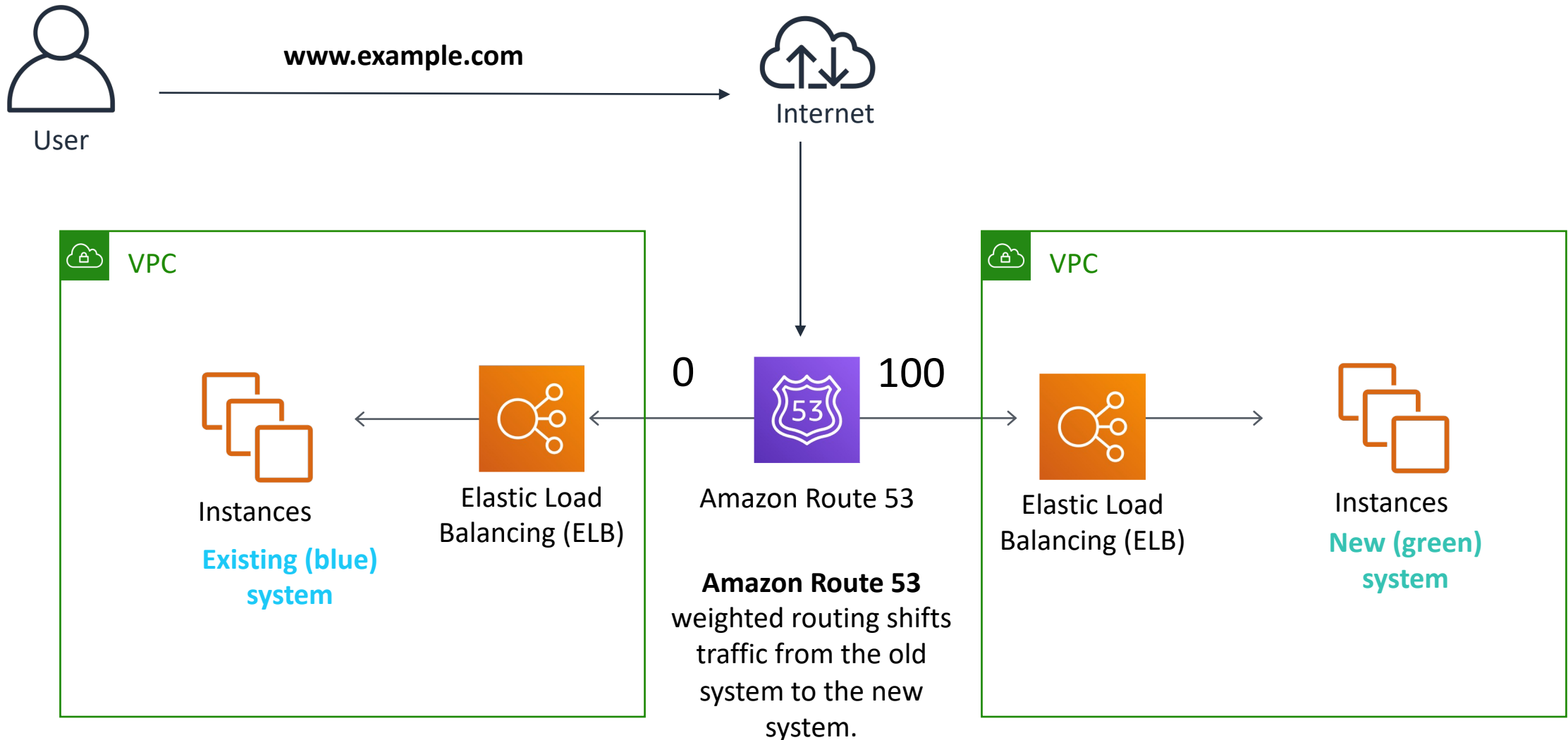
5. Geolocation routing policy (DNS query location)
6. Geoproximity routing policy (traffic flow to AWS Region, or latitude and longitude)
7. Multivalue answer routing policy

# **Recorded demo: Amazon Route 53 routing options**

# Latency-based routing (LBR)



# Example: Blue/green deployment



## Testing scalability:

- Find problems earlier rather than later
- Simulate traffic with load testing
- Numerous open source tools are available:
  - [TheGrinder](#)
  - [Apache Jmeter](#)
  - [Bees with Machine Guns](#): Simple load testing script that's specific to Amazon EC2

## Key Takeaway



- Amazon Route 53 supports multiple different routing policies.
- Amazon Route53 helps:
  - Register / transfer domain names.
  - Resolve domain names to IP address.
- Amazon Route53 supports multiple routing policies.
- Amazon Route53 can route traffic to resources in different AWS regions as well as on premises resources.
- Latency-based routing (LBR) enables DNS to route user requests to the AWS Region with the fastest response.

- Elastic Load Balancing (ELB) features
- Types of ELB load balancers
- Amazon EC2 Auto Scaling and launch configurations
- Amazon Route 53 features
- Failover routing configuration

**To finish this module:**

Complete:  **Knowledge Check**

# Test Strategies



# Module 4 Scenario:

---

A floral warehouse has just expanded delivery of wholesale flowers and plants to another region of the country and has experienced unexpected growth. They have an online ordering system that promises 48 hour delivery; therefore, they need quick notification of orders to the fulfillment center and a way to track their inventory more efficiently. As a consultant working with a floral warehouse customer that wants to enhance the infrastructure to support the growth, but needs to meet compliance requirements.

---

What type of load balancer would provide the most benefit to the desired infrastructure?



# Module 4 Scenario:

---

A floral warehouse has just expanded delivery of wholesale flowers and plants to another region of the country and has experienced unexpected growth. They have an online ordering system that promises 48 hour delivery; therefore, they need quick notification of orders to the fulfillment center and a way to track their inventory more efficiently. As a consultant working with a floral warehouse customer that wants to enhance the infrastructure to support the growth, but needs to meet compliance requirements.

---

What type of load balancer would provide the most benefit to the desired infrastructure?



# Module 4 Scenario:

---

A floral warehouse has just expanded delivery of wholesale flowers and plants to another region of the country and has experienced unexpected growth. They have an online ordering system that promises 48 hour delivery; therefore, they need quick notification of orders to the fulfillment center and a way to track their inventory more efficiently. As a consultant working with a floral warehouse customer that wants to enhance the infrastructure to support the growth, but needs to meet compliance requirements.

---

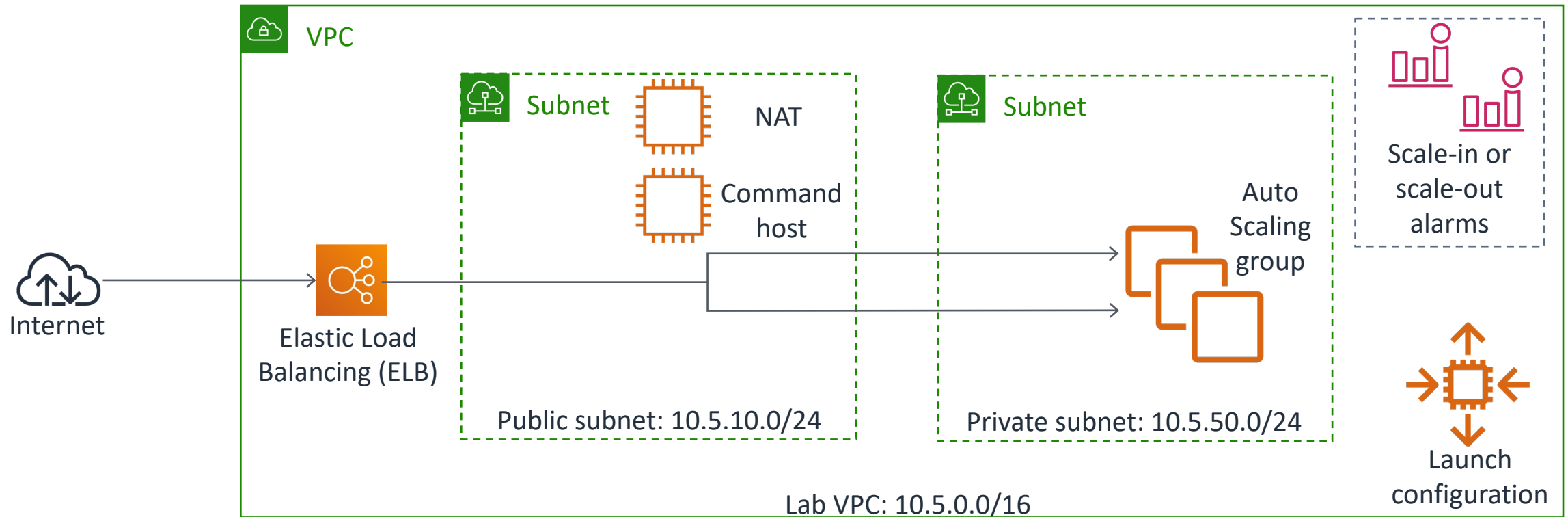
What type of load balancer would provide the most benefit to the desired infrastructure?

- A. Application
- B. Classic
- C. Elastic
- D. All of the above
- E. None of the above

# Lab 3: Configure EC2 Auto Scaling



# Lab 3: Configure EC2 Auto Scaling

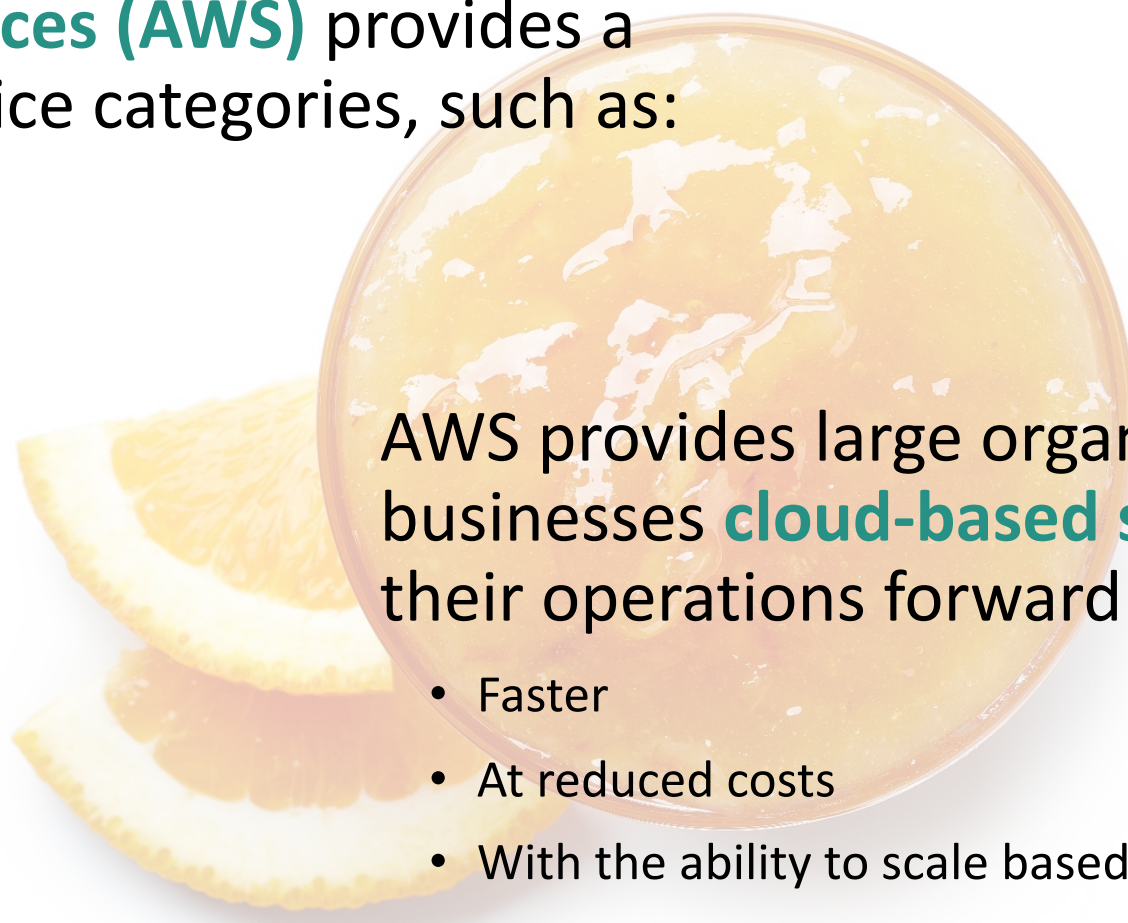


Systems Operations Project Scenario

# Mom & Pop Café & Bakery

**Amazon Web Services (AWS)** provides a broad array of service categories, such as:

- Compute
- Storage
- Database
- Analytics

A background image featuring a glass of orange juice with a slice of orange floating in it, and two more orange slices on the surface in front of the glass.

AWS provides large organizations and small businesses **cloud-based solutions** to move their operations forward:

- Faster
- At reduced costs
- With the ability to scale based on demand

On completion of this project, you will be able to:

- **Experience** the communication challenges faced when attempting to apply technology as the solution to business problems.
- **Translate** customer requirements into a proposed technical solution.
- **Present** the proposed solution to the customer.
- **Use** AWS Best Practices for a frugal, resilient, and secure solution.



# Mom & Pop Café & Bakery

## Customer Meeting Preparation

Mom & Pop Café & Bakery

# Request to expand the website internationally

# Business need: Website expansion



Phillip is excited—his second store is about to open next month. It will be in the US state of Virginia, near Emma's cousin.

This time, Philip decides to have the website ready to go for opening day. He asks Sophie to expand the café's global website presence.

# Going international with Amazon Route 53



Matt offers Sophie a Playbook that she can use for troubleshooting.

Raquel also refers her to a website for developing their own Playbook.

Learn more about  
Playbooks

# Troubleshooting knowledge base

Sophie and Darsh decide to use Matt's troubleshooting knowledge base to add failover routing to their current website, using AWS best practices.



# DEMO: GEOROUTING

# Geographic queries

How would end users be routed to the closest geographic location?

Resolution #1

Resolution #2

Resolution #3

# Geographic queries

How would end users be routed to the closest geographic location?

## Resolution #1

Create a Route  
53 Hosted Zone.

## Resolution #2

Create a record  
set.

## Resolution #3

Use a  
geolocation  
routing policy.

# Routing policy

**Which routing policy allows for traffic to be routed based on the physical location of the client user?**

**Answer:**

# Routing policy

Which routing policy allows for traffic to be routed based on the physical location of the client user?

Answer:

Geoproximity routing policy

# Activity 4: Configuring Failover Routing with Route 53



# Project time: Troubleshooting knowledge base



# Thank you

© 2019 Amazon Web Services, Inc. or its affiliates. All rights reserved. This work may not be reproduced or redistributed, in whole or in part, without prior written permission from Amazon Web Services, Inc. Commercial copying, lending, or selling is prohibited. Corrections or feedback on the course, please email us at: [aws-course-feedback@amazon.com](mailto:aws-course-feedback@amazon.com). For all other questions, contact us at: <https://aws.amazon.com/contact-us/aws-training/>. All trademarks are the property of their owners.

